Laboratory Work 4
SQL Queries, Functions, and Operators

Part 1: Basic SELECT Queries

```
-- Task 1.1
SELECT CONCAT(first_name, ' ', last_name) AS full_name,
department, salary
FROM employees;

-- Task 1.2
SELECT DISTINCT department
FROM employees;

-- Task 1.3
SELECT project_name, budget,
CASE
WHEN budget > 150000 THEN 'Large'
WHEN budget BETWEEN 100000 AND 150000 THEN 'Medium'
ELSE 'Small'
END AS budget_category
FROM projects;

-- Task 1.4
SELECT CONCAT(first_name, ' ', last_name) AS full_name,
COALESCE(email, 'No email provided') AS email
FROM employees;
```

Part 2: WHERE Clause and Comparison Operators

```
-- Task 2.1
SELECT *
FROM employees
WHERE hire_date > '2020-01-01';

-- Task 2.2
SELECT *
FROM employees
WHERE salary BETWEEN 60000 AND 70000;

-- Task 2.3
SELECT *
FROM employees
WHERE last_name LIKE 'S%' OR last_name LIKE 'J%';

-- Task 2.4
SELECT *
FROM employees
WHERE manager_id IS NOT NULL
AND department = 'IT';
```

Part 3: String and Mathematical Functions

```
-- Task 3.1
SELECT UPPER(CONCAT(first_name, ' ', last_name)) AS full_name,
LENGTH(last_name) AS last_name_length,
SUBSTRING(email FROM 1 FOR 3) AS email_start
FROM employees;

-- Task 3.2
SELECT first_name, last_name,
salary AS annual_salary,
ROUND(salary / 12, 2) AS monthly_salary,
salary * 0.10 AS raise_amount
FROM employees;

-- Task 3.3
SELECT FORMAT('Project: %s - Budget: $%s - Status: %s',
project_name, budget, status) AS project_info
FROM projects;

-- Task 3.4
SELECT first_name, last_name,
EXTRACT(YEAR FROM AGE(CURRENT_DATE, hire_date)) AS years_worked
FROM employees;
```

Part 4: Aggregate Functions and GROUP BY

```
-- Task 4.1
SELECT department, AVG(salary) AS average_salary
FROM employees
GROUP BY department;

-- Task 4.2
SELECT p.project_name,
COALESCE(SUM(a.hours_worked), 0) AS total_hours
FROM projects p
LEFT JOIN assignments a ON p.project_id = a.project_id
GROUP BY p.project_id, p.project_name;

-- Task 4.3
SELECT department, COUNT() AS employee_count
FROM employees
GROUP BY department
HAVING COUNT() > 1;

-- Task 4.4
SELECT MAX(salary) AS maximum_salary,
MIN(salary) AS minimum_salary,
SUM(salary) AS total_payroll
FROM employees;
```

Part 5: Set Operations

```
-- Task 5.1
SELECT employee_id,
CONCAT(first_name, ' ', last_name) AS full_name,
salary
FROM employees
WHERE salary > 65000

UNION

SELECT employee_id,
CONCAT(first_name, ' ', last_name) AS full_name,
salary
FROM employees
WHERE hire_date > '2020-01-01';

-- Task 5.2
SELECT employee_id
FROM employees
WHERE department = 'IT'

INTERSECT

SELECT employee_id
FROM employees
WHERE salary > 65000;

-- Task 5.3
SELECT employee_id
FROM employees

EXCEPT

SELECT employee_id
FROM assignments;
```

Part 6: Subqueries

```
-- Task 6.1
SELECT *
FROM employees e
WHERE EXISTS (
SELECT 1
FROM assignments a
WHERE a.employee_id = e.employee_id
);

-- Task 6.2
SELECT *
FROM employees
WHERE employee_id IN (
SELECT a.employee_id
FROM assignments a
```

```
JOIN projects p ON a.project_id = p.project_id
WHERE p.status = 'Active'
);

-- Task 6.3
SELECT *
FROM employees
WHERE salary > ANY (
SELECT salary
FROM employees
WHERE department = 'Sales'
);
```

Part 7: Complex Queries

```
-- Task 7.1
SELECT CONCAT(e.first_name, ' ', e.last_name) AS employee_name,
e.department,
ROUND(AVG(a.hours_worked), 2) AS average_hours,
RANK() OVER (
PARTITION BY e.department
ORDER BY e.salary DESC
) AS salary_rank
FROM employees e
LEFT JOIN assignments a ON e.employee_id = a.employee_id
GROUP BY e.employee_id, e.first_name, e.last_name,
e.department, e.salary;

-- Task 7.2
SELECT p.project_name,
SUM(a.hours_worked) AS total_hours,
COUNT(DISTINCT a.employee_id) AS employee_count
FROM projects p
JOIN assignments a ON p.project_id = a.project_id
GROUP BY p.project_id, p.project_name
HAVING SUM(a.hours_worked) > 150;

-- Task 7.3
SELECT e.department,
COUNT(*) AS total_employees,
ROUND(AVG(e.salary), 2) AS average_salary,
MAX(e.salary) AS highest_salary,
CONCAT(m.first_name, ' ', m.last_name) AS highest_paid_employee,
GREATEST(MAX(e.salary), MIN(e.salary)) AS greatest_salary,
LEAST(MAX(e.salary), MIN(e.salary)) AS lowest_salary
FROM employees e
JOIN employees m ON m.employee_id = (
SELECT e2.employee_id
FROM employees e2
WHERE e2.department = e.department
ORDER BY e2.salary DESC, e2.employee_id
LIMIT 1
```

```
)
GROUP BY e.department, m.first_name, m.last_name;
```